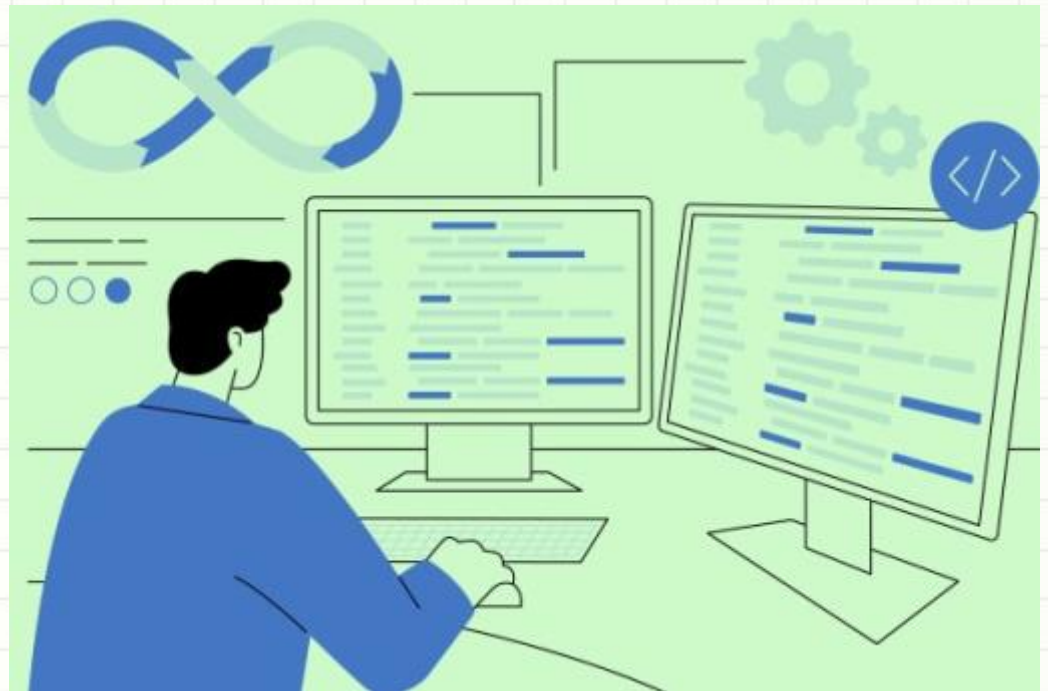


HỆ THỐNG GIÁM SÁT TOÀN DIỆN (OBSERVABILITY STACK)



Mục tiêu buổi học

- 1) Hiểu sự khác nhau giữa monitoring và observability
- 2) Hiểu 3 cột trụ trong observability: logs (nhật ký), metrics (chỉ số) và traces (truy vết)
- 3) Hiểu 4 tín hiệu vàng: latency (độ trễ), traffic (lưu lượng), errors (lỗi), saturation (bão hòa)
- 4) Xây dựng hệ thống giám sát toàn diện



Nội dung

1

Monitoring vs Observability

2

3 cột trụ: Logs, Metrics, Traces

3

4 tín hiệu vàng: Latency, Traffic, Errors, Saturation

4

Observability Stack

Monitoring vs Observability

Vấn đề thực tế sau khi ra production:

- user có thể gặp lỗi
- server có thể quá tải
- API có thể chậm
- database có thể nghẽn

Nếu không theo dõi sẽ không biết hệ thống hoạt động ra sao

Monitoring vs Observability

Ví dụ thực tế : website chạy chậm. Nguyên nhân có thể:

- CPU server quá tải
- database query chậm
- API bị lỗi
- network latency cao

Làm sao để xác định đúng nguyên nhân?

=> Cần hệ thống giám sát

Monitoring vs Observability

Mục tiêu của hệ thống giám sát là gì?

Cung cấp dữ liệu quan trọng để duy trì hiệu suất, tính khả dụng và chuyển đổi các hoạt động từ thể bị động ("chữa cháy") sang chủ động (dự báo và ngăn chặn)

Monitoring vs Observability

Monitoring:

- Là việc **theo dõi các chỉ số đã biết trước** (CPU, RAM, latency, error rate...).
- Dựa trên **metrics + alert** để phát hiện hệ thống có bất thường.
- Trả lời câu hỏi: “**Hệ thống có đang hoạt động bình thường không?**”
- Phù hợp khi ta đã biết cần theo dõi gì
- Ví dụ: cảnh báo khi CPU > 80% hoặc lỗi 5xx tăng cao

Monitoring vs Observability

Observability:

- Là khả năng **hiểu nguyên nhân bên trong hệ thống** thông qua dữ liệu giám sát
- Kết hợp **logs + metrics + traces** để phân tích sự cố phức tạp
- Trả lời câu hỏi: “**Tại sao hệ thống gặp vấn đề?**”
- Hữu ích trong hệ thống phức tạp như microservices.
- Ví dụ: truy vết request qua nhiều service để tìm bottleneck

3 cột trụ: Logs, Metrics và Traces



3 cột trụ: Logs, Metrics và Traces

1. Logs (Nhật ký):

Định nghĩa: Là dữ liệu về các sự kiện riêng lẻ xảy ra

Đặc điểm: chứa thông tin phong phú, thường ở dạng văn bản tự do hoặc có cấu trúc (JSON) để có thể đọc được. Logs có kích thước lớn hơn nhiều so với metrics, có thể gây ra các vấn đề về xử lý nếu ghi quá chi tiết

Mục đích: Cung cấp thông tin chi tiết để hiểu "tại sao" một sự kiện hoặc lỗi cụ thể lại xảy ra. Khi metrics cảnh báo về một nguồn vấn đề tiềm ẩn, logs sẽ được sử dụng để phân tích sâu hơn về những gì đã thực sự diễn ra tại thời điểm đó

Ví dụ: Thông báo lỗi giao dịch thất bại kèm theo ID người dùng, bản ghi đăng nhập hệ thống, hoặc các dòng thông báo từ container

3 cột trụ: Logs, Metrics và Traces

2. Metrics (Chỉ số đo lường):

Định nghĩa: Là các phép đo tại một thời điểm nhất định đại diện cho trạng thái của hệ thống

Đặc điểm: Dữ liệu metrics có kích thước nhỏ, cho phép thu thập hiệu quả ở quy mô lớn mà không tốn nhiều chi phí xử lý và lưu trữ. Thường được gắn thẻ (tagged) để dễ dàng nhóm và tìm kiếm

Mục đích: Thông báo sức khỏe và hoạt động của các thành phần hệ thống. Là nguồn dữ liệu cho việc thiết lập các cảnh báo tự động (alerting) vì chúng phản ánh nhanh chóng các ngưỡng bất thường

Ví dụ: HTTP request mỗi giây, tỷ lệ lỗi 5xx, mức độ sử dụng CPU hoặc bộ nhớ, độ trễ P95

3 cột trụ: Logs, Metrics và Traces

3. Traces (Truy vết):

Định nghĩa: Cung cấp một cái nhìn liên tục về luồng xử lý và sự tiến triển của dữ liệu trong ứng dụng

Đặc điểm: Đại diện cho hành trình của một người dùng duy nhất đi qua toàn bộ ứng dụng. Mỗi trace cần có một định danh duy nhất (unique identifier) để liên kết các bước xử lý lại với nhau

Mục đích: Tối ưu hóa hiệu năng và xác định nút thắt cổ chai. Traces giúp xác định chính xác nơi nào đang gây ra độ trễ

Ví dụ: Theo dõi một yêu cầu từ trình duyệt, đi qua API Gateway, đến dịch vụ thanh toán và cuối cùng là ghi vào cơ sở dữ liệu

3 cột trụ: Logs, Metrics và Traces

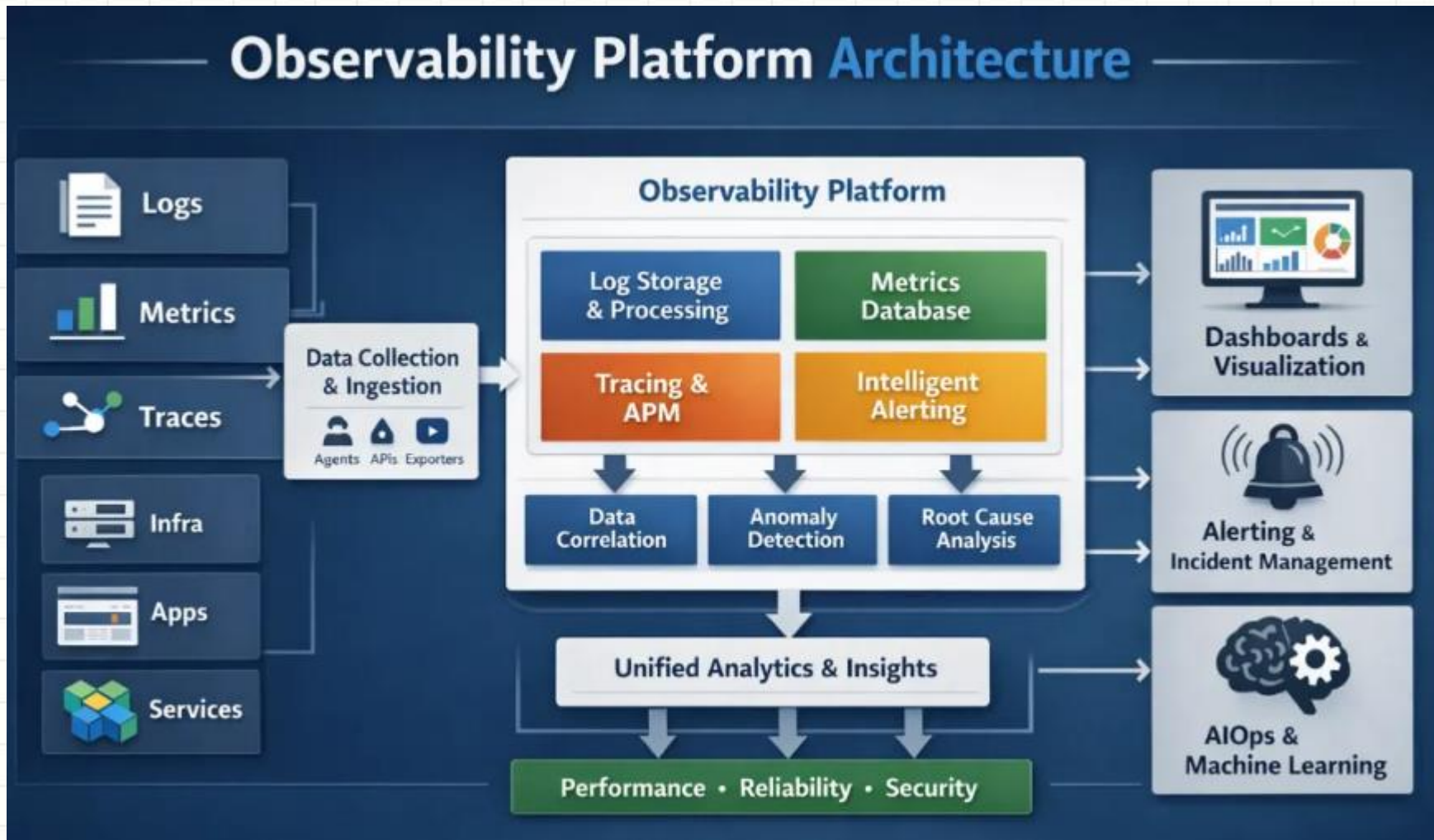
Telemetry:

Những dữ liệu về logs, metrics, traces gọi chung là dữ liệu telemetry

OpenTelemetry (OTel):

Là bộ công cụ và tiêu chuẩn, giúp ứng dụng tạo ra dữ liệu telemetry và chuyển đến nơi xử lý. Hiểu nôm na OpenTelemetry như là một cảm biến gắn vào ứng dụng, giúp ứng dụng gửi dữ liệu telemetry về trung tâm giám sát

3 cột trụ: Logs, Metrics và Traces



4 tín hiệu vàng

1. Latency (Độ trễ)

Định nghĩa: Là thời gian cần thiết để phục vụ một yêu cầu. Nên tập trung vào độ trễ **P95 hoặc P99** thay vì giá trị trung bình.

- **P95/P99** có nghĩa là 5% hoặc 1% yêu cầu của người dùng đang mất quá nhiều thời gian để hoàn thành.
Vd: P95=800ms nghĩa là 5% request mất hơn 800ms để hoàn thành
- Việc theo dõi số trung bình thường gây hiểu lầm vì một lượng lớn các yêu cầu nhanh có thể che lấp đi một vài yêu cầu cực chậm, làm ẩn đi vấn đề thực tế mà người dùng đang gặp phải,.

Dấu hiệu: Sự gia tăng dần dần là chỉ số sớm cho thấy sự suy giảm hiệu năng do thắt cổ chai cơ sở dữ liệu hoặc mã nguồn không tối ưu

4 tín hiệu vàng

2. Traffic (Lưu lượng)

Định nghĩa: Đo lường mức độ nhu cầu đặt lên hệ thống. Thường dùng các metrics: requests per seconds, transactions per seconds, active users

Ví dụ: Số lượng yêu cầu HTTP mỗi giây hoặc thông lượng băng thông đối với một bộ cân bằng tải

Dấu hiệu: Một sự sụt giảm lưu lượng đột ngột và không giải thích được có thể chỉ ra lỗi nghiêm trọng từ phía thượng nguồn (upstream), chẳng hạn như vấn đề về DNS hoặc lỗi bộ cân bằng tải khiến người dùng không thể truy cập dịch vụ

4 tín hiệu vàng

3. Errors (Lỗi)

Định nghĩa: Tỷ lệ các yêu cầu thất bại, có thể là lỗi rõ ràng (như HTTP 5xx), lỗi ngầm định (trả về kết quả sai mặc dù mã trạng thái là 200), hoặc lỗi do vi phạm chính sách

Cách đo lường hiệu quả: Cảnh báo không nên chỉ dựa trên số lượng lỗi tuyệt đối mà nên dựa trên **tỷ lệ lỗi so với tổng số yêu cầu**. Ví dụ: Thiết lập cảnh báo khi 5% số yêu cầu dẫn đến lỗi máy chủ trong một khoảng thời gian 5 phút.

4 tín hiệu vàng

4. Saturation (Mức độ bão hòa)

Định nghĩa: Đo lường mức độ "đầy" của các tài nguyên trong hệ thống hoặc dịch vụ. Nó cho biết hệ thống còn bao nhiêu tài nguyên để có thể phục vụ thêm các yêu cầu mới

Các chỉ số chính: CPU usage, RAM usage, Disk usage, và các tài nguyên giới hạn khác như **Connection Pool** của cơ sở dữ liệu,

Dấu hiệu cảnh báo: Khi các chỉ số này liên tục vượt ngưỡng cao (ví dụ: 85% đến 90%), đó là tín hiệu cho thấy hệ thống sắp đạt đến giới hạn và có khả năng bắt đầu điều tiết hoặc bị treo. Ví dụ: một cảnh báo bão hòa như "Dự báo sử dụng đĩa" giúp đội ngũ DevOps biết trước khi nào đĩa sẽ đầy (ví dụ: trong 48 giờ tới) dựa trên tốc độ tiêu thụ hiện tại để xử lý chủ động

Observability stack

- 1) Xác định yêu cầu và mục tiêu
- 2) Chọn lựa công cụ (stack): stack mã nguồn mở, stack trả phí, kết hợp cả hai
- 3) Khai triển và tích hợp vào ứng dụng
- 4) Xây dựng dashboard giám sát và các cảnh báo
- 5) Duy trì và tối ưu hệ thống giám sát

Observability stack

